



UNIDAD #12:

Tabla de símbolos

**Memoria semántica para identificadores,
tipos, ámbitos y errores**

Unidad 12



Agenda

- 1 Concepto de tabla de símbolos
- 2 Información que almacena
- 3 Operaciones principales
- 4 Ámbitos y alcance
- 5 Errores semánticos comunes
- 6 Ejemplo práctico y actividad

PROPÓSITO

Competencia a desarrollar



- Comprender la función de la tabla de símbolos dentro del análisis semántico.
- Modelar identificadores, tipos, roles y ámbitos usando una estructura organizada.
- Aplicar la tabla de símbolos para detectar errores semánticos en un mini lenguaje.



FORMACIÓN

Valor de la semana

- Responsabilidad: registrar correctamente los identificadores evita errores ocultos en el intérprete o compilador.
- Ética académica: comprender la lógica interna permite defender el código y explicar las decisiones de implementación.

RESULTADO ESPERADO

Objetivo de la unidad



- Al finalizar la sesión, el estudiante será capaz de diseñar una tabla de símbolos básica.
- La tabla permitirá registrar variables, funciones y parámetros con su tipo y ámbito.
- El estudiante usará la estructura para reportar errores semánticos comunes.

CONTENIDO

¿Qué es una tabla de símbolos?



- Es una estructura de datos que almacena información de los identificadores de un programa.
- Funciona como una memoria semántica consultada durante la compilación o interpretación.
- Permite validar si una variable, función, parámetro o clase fue declarada correctamente.

CONTENIDO

¿Qué información almacena?



Identificador

- Nombre del símbolo.
- Tipo de dato.
- Rol dentro del programa.
- Línea y columna de declaración.

Contexto

- Ámbito donde fue declarado.
- Lista de parámetros, si es función.
- Valor inicial, si aplica.
- Referencia al entorno padre.

CONTENIDO

Estructura de una entrada

Campo	Ejemplo	Uso dentro del compilador
nombre	contador	Identificador declarado
tipo	int	Validar operaciones y asignaciones
rol	variable	Distinguir variable, función o parámetro
ámbito	función main	Resolver visibilidad y alcance
línea / columna	12 : 5	Reportar errores con ubicación



FUNCIONAMIENTO

Operaciones principales

insertar()

Registra un nuevo símbolo cuando aparece una declaración.

buscar()

Encuentra un símbolo por nombre desde el ámbito actual.

actualizar()

Completa datos como tipo, rol, valor o parámetros.

abrirÁmbito()

Crea un entorno hijo para una función, clase o bloque.

cerrarÁmbito()

Regresa al entorno padre al terminar un bloque.



MODELO MENTAL

Ámbitos y alcance

Global

- pi : float
- saludo : string

Función calcular()

- x : int
- y : int

Bloque if

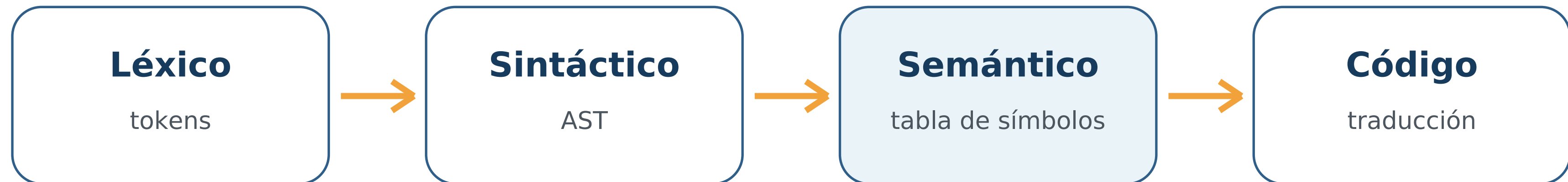
- temp : int



Búsqueda: ámbito actual -> ámbito padre -> global

FASES DEL COMPILADOR

¿Dónde se usa?



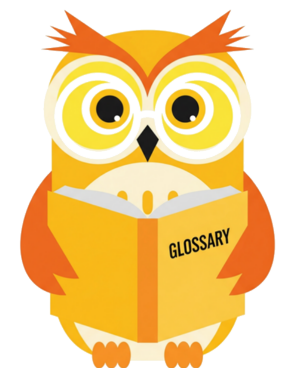
La tabla de símbolos se consulta en el análisis semántico para validar declaraciones, tipos y alcance.



SEMÁNTICA

Errores que ayuda a detectar

- **Variable no declarada** usar x antes de declararla
- **Redeclaración inválida** declarar contador dos veces en el mismo ámbito
- **Tipo incompatible** asignar string a una variable int
- **Llamada incorrecta** enviar 3 argumentos a una función que recibe 2
- **Retorno inválido** devolver string en una función int



IMPLEMENTACIÓN

Ejemplo en TypeScript



```
type Rol = "variable" | "funcion" | "parametro";

class Simbolo {
  constructor(
    public nombre: string,
    public tipo: string,
    public rol: Rol,
    public linea: number,
    public columna: number
  ) {}
}

class Entorno {
  private tabla = new Map<string, Simbolo>();
  constructor(public nombre: string,
    public anterior?: Entorno) {}
}
```

IMPLEMENTACIÓN

Búsqueda en cadena

Método buscar(nombre)

```
buscar(nombre: string): Simbolo | null {  
  let actual: Entorno | undefined = this;  
  
  while (actual) {  
    const simbolo = actual.tabla.get(nombre);  
    if (simbolo) return simbolo;  
    actual = actual.anterior;  
  }  
  
  return null;  
}
```

Si no está en el entorno actual, se busca en el entorno padre.





Ejemplo práctico

Construcción de tabla de símbolos desde un AST

- Recorrer declaraciones de variables y funciones.
- Insertar cada identificador en el entorno correcto.
- Validar redeclaraciones y usos no declarados.
- Mostrar la tabla final con nombre, tipo, rol y ámbito.



Practiquemos lo aprendido

Actividad individual o en parejas

- Dado un fragmento de código, identificar todos los símbolos.
- Separar símbolos por ámbito: global, función y bloque.
- Marcar errores: redeclaración, variable no declarada o tipo incompatible.
- Entregar una tabla con: nombre, tipo, rol, ámbito y línea.

Conceptos clave aprendidos

Símbolo: Representa un identificador del programa.

Tabla: Estructura que guarda los símbolos conocidos.

Ámbito: Zona donde un símbolo es visible.

Shadowing: Un símbolo local puede ocultar a uno externo.

Error semántico: El código tiene forma válida, pero significado incorrecto.



Referencias

- Aho, A., Lam, M., Sethi, R. & Ullman, J. (2007). Compilers: Principles, Techniques & Tools.
- Loudon, K. (2004). Lenguajes de programación: principios y práctica.
- Parr, T. (2010). Language Implementation Patterns.
- Material de clase: Organización de Lenguajes y Compiladores 1.





DUDAS ¿?



**¡GRACIAS POR SU
ATENCIÓN!**

Recuerda consultar el foro semanal para resolver dudas de la unidad.